

ChromaFlow: A Negative Ablation Study of Orchestration Overhead in Tool-Augmented Agent Evaluation

Tarun Mittal
Octave-X

May 13, 2026

Abstract

Autonomous language-model agents increasingly combine planning, tool use, document processing, browsing, code execution, and verification loops. These capabilities make agent systems more useful, but they also introduce operational failure modes that are not visible from final accuracy alone. This report presents ChromaFlow, a tool-augmented autonomous reasoning framework built around planner-directed execution, specialized tool use, and telemetry-driven evaluation. We analyze ChromaFlow on GAIA 2023 Level-1 validation tasks under clean evaluation constraints. A frozen full Level-1 baseline achieved 29/53 correct answers, or 54.72%. A later recovery configuration with expanded orchestration achieved 27/53 correct answers, or 50.94%, while increasing tracebacks, timeout events, tool-failure mentions, token-line calls, and campaign-log cost estimates. Two randomized 20-task smoke evaluations produced 12/20 and 11/20 correct answers, showing that small diagnostic gains can be unstable across samples. The central result is therefore a negative ablation: more aggressive orchestration did not improve full-set performance and increased operational noise. The report argues that bounded planner escalation, deterministic extraction, evidence reconciliation, and explicit run gates should be treated as first-order requirements for reliable autonomous agent evaluation.

Keywords: autonomous agents, tool-augmented reasoning, multi-agent systems, GAIA, benchmark evaluation, reliability, negative ablation

1 Introduction

Large language model systems have moved from single-turn text generation toward autonomous execution loops that plan, call tools, inspect intermediate evidence, and revise answers. Systems such as ReAct showed the value of interleaving reasoning and actions, while tool-use work such as Toolformer demonstrated that language models can benefit from external APIs and structured tool calls [2, 3]. Benchmark suites such as GAIA measure longer-horizon assistant behavior that often requires search, file handling, multimodal reasoning, and careful answer synthesis [1]. Since GAIA, agent evaluation has broadened toward multi-environment agent benchmarks, realistic web and software-engineering tasks, operating-system interaction, enterprise workflows, tool-agent-user interaction, and explicit orchestration benchmarks [4, 5, 6, 7, 8, 9, 11]. Recent agent-evaluation and agent-skills work also emphasizes reliability, contamination, evolution, governance, and failure modes beyond raw tool use [12, 10]. This paper is positioned within that evaluation-methodology line: it studies when additional orchestration fails to produce a reliable improvement.

These systems are frequently evaluated by final task accuracy. Accuracy is necessary, but it is incomplete. A configuration can preserve or even improve accuracy while becoming more expensive,

slower, harder to reproduce, or more dependent on brittle retry loops. Conversely, a negative result can still be useful if it exposes which architectural choices increase execution instability. This paper treats operational telemetry as part of the evaluation object rather than as incidental logging.

ChromaFlow is an autonomous reasoning framework designed for planner-directed tool use and benchmark execution. It supports document analysis, web search, code execution, structured extraction, browser interaction, and final-answer verification. The system was developed to study whether adaptive orchestration and reliability hardening can improve benchmark performance under clean evaluation constraints.

The main finding is deliberately conservative. A smaller 20-task diagnostic signal suggested that targeted reliability fixes could help. However, a later full Level-1 recovery run did not preserve the improvement. The recovery configuration scored lower than the frozen baseline and generated more operational noise. A subsequent randomized smoke run also failed the gate for a new full run. We therefore present the work as a negative ablation and a reliability analysis, not as a leaderboard claim.

2 Contributions

This report makes three focused contributions:

- It documents a negative ablation of autonomous-agent orchestration in which a more aggressive recovery configuration reduced full GAIA Level-1 accuracy while increasing operational noise and cost estimates.
- It provides a clean-evaluation reporting protocol that preserves benchmark integrity by publishing aggregate metrics while withholding task text, task identifiers, gold answers, predictions, attachments, raw URLs, and raw traces.
- It proposes reliability gates for agent benchmarks: smoke-run gains should justify full runs only when they clear both accuracy and operational-noise thresholds under randomized sampling.

This paper does not claim state-of-the-art performance on GAIA. Instead, it contributes a negative systems ablation: a more aggressive orchestration configuration decreased full-set Level-1 accuracy while increasing tracebacks, timeout mentions, tool-failure mentions, and cost estimates. The result supports a reliability-centered evaluation protocol in which agent changes must pass both accuracy and operational-noise gates before being treated as progress.

3 Related Work

3.1 Reasoning, acting, and tool use

ReAct established a simple but influential pattern for interleaving reasoning traces and task-specific actions, allowing the model to update plans while interacting with external information sources [2]. Toolformer showed that language models can be trained to decide when and how to call external APIs, framing tool use as a learned language-model behavior rather than a fixed wrapper around generation [3]. These systems motivate tool-augmented agents, but they do not by themselves resolve the operational question studied here: when does extra tool use or planning increase reliability, and when does it merely amplify noise?

3.2 Agent benchmarks and realistic environments

GAIA evaluates general assistant behavior across search, file handling, multimodal evidence, and concise answer synthesis [1]. AgentBench broadened the evaluation of language models as agents across multiple interactive environments [4]. WebArena and WorkArena moved evaluation toward realistic browser and enterprise-software tasks [5, 8], while SWE-bench evaluated software-engineering agents on real GitHub issue resolution [6]. OSWorld further emphasized open-ended interaction with real computer environments [7]. Tau-bench added dynamic tool-agent-user interaction with domain-specific APIs and policies [9]. Relative to these benchmarks, ChromaFlow’s contribution is not a new public task suite; it is an operational analysis of a GAIA configuration change that looked promising in smoke tests but failed under a full Level-1 comparison.

3.3 Orchestration and evaluation reliability

The 2025–2026 agent literature increasingly treats orchestration as a measurable intervention rather than an unqualified good. OrchestrationBench explicitly targets LLM-driven planning and tool use across multi-domain scenarios [11]. Recent systematization and evaluation work argues that agentic behavior extends beyond isolated tool calls and raises reliability, contamination, evolution, governance, and security questions [10, 12]. This paper provides a concrete case study supporting that view: the recovery configuration added planning and execution surface area, but the full-run result degraded accuracy while increasing operational noise.

4 System Overview

ChromaFlow is organized around a supervisory execution controller that selects an execution strategy, routes tool calls, monitors failure signals, and synthesizes final answers. The controller may use direct answer synthesis for simple tasks, or it may allocate work across specialized execution paths for retrieval, document analysis, browser interaction, Python execution, shell execution, and verification.

Figure 1 summarizes the system topology. The diagram uses a separated control plane, execution plane, evidence layer, and synthesis layer to emphasize that ChromaFlow’s reliability policy is not a single component; it is a constraint applied across routing, tool execution, and final-answer assembly.

4.1 Planner-directed execution

Incoming tasks are mapped to a task signature that estimates modality, attachment requirements, likely answer shape, and tool needs. The planner then chooses an execution path and a tool budget. The goal is to avoid treating every question as a broad web-research problem. Tasks with documents, spreadsheets, images, or code artifacts should first extract local evidence before escalating to search-heavy workflows.

4.2 Tool layer

The tool layer includes web search, document parsing, Python execution, shell execution, browser automation, media handling, and final answer normalization. Tool outputs are treated as evidence objects that can be inspected by the answering policy. The system tracks tool exceptions, timeouts, fallback activation, and retry behavior because those events often predict answer quality and cost.

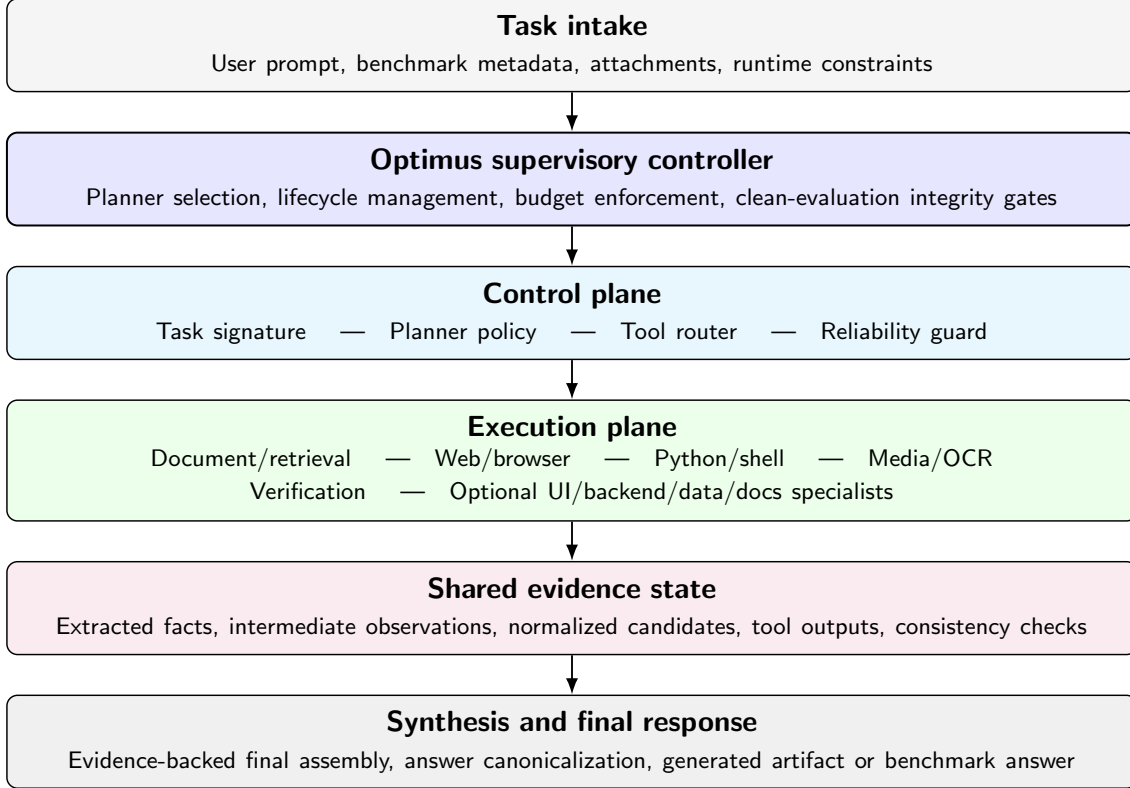


Figure 1: Professionalized ChromaFlow system topology. The control plane profiles the task, chooses an execution policy, routes tools, and enforces reliability gates. The execution plane writes observations into a shared evidence state before synthesis produces the final response. The layered layout keeps routing explicit without overlapping edges or labels.

4.3 Reliability layer

The reliability layer is responsible for bounding tool calls, detecting retry storms, handling timeouts, and preventing unavailable optional providers from generating repeated tracebacks. It also records missing finals, direct-solver integrity hits, elapsed time, attempts, and aggregate cost estimates. These signals are used as run gates: a configuration should not graduate from smoke testing to a full run unless it improves accuracy without materially increasing operational noise.

5 Evaluation Protocol

The evaluation used GAIA 2023 Level-1 validation tasks. To preserve benchmark integrity, public reporting excludes task text, task identifiers, gold answers, model predictions, attachment names, raw URLs, and raw execution traces. Only aggregate accuracy, aggregate operational metrics, and non-identifying transition counts are reported.

The clean evaluation constraints were:

- direct solver paths disabled;
- no task-specific answer patches or task-ID routing;
- no publication of benchmark task text or gold answers;

- zero tolerance for missing final answers in completed full runs;
- run directories frozen for later audit;
- full-run launch gated by randomized smoke-run performance and noise.

The primary comparison is between a frozen full Level-1 baseline and a later full Level-1 recovery configuration. Both runs covered 53 Level-1 validation tasks. Additional 20-task randomized smoke runs were used as diagnostic gates, not as proof of full-set performance.

6 Results

6.1 Full Level-1 comparison

Table 1 summarizes the full Level-1 comparison. The frozen baseline achieved 29/53 correct answers. The recovery configuration achieved 27/53 correct answers. Both runs produced zero missing finals.

Table 1: Full GAIA Level-1 validation comparison.

Metric	Frozen baseline	Recovery configuration
Correct / total	29 / 53	27 / 53
Accuracy	54.72%	50.94%
Elapsed time	6653.89 s	7230.93 s
Average task time	115.86 s	124.89 s
Total attempts	57	58
Missing finals	0	0
Seed	1782047163	1778025467

The recovery configuration therefore produced a net loss of two tasks. This is not an improvement claim. It is evidence that the expanded orchestration configuration failed to generalize from smaller diagnostic signals to the full Level-1 task set.

6.2 Operational noise and cost

Table 2 reports aggregate operational telemetry. The recovery configuration increased tracebacks, timeout mentions, tool-failure mentions, and campaign-log cost estimates.

Table 2: Operational noise and campaign-log cost estimates. Cost estimates are operational estimates, not provider billing statements.

Metric	Frozen baseline	Recovery configuration
Tracebacks	62	141
Timeout mentions	769	939
Tool-failure mentions	170	282
Priority cost estimate	\$197.32	\$231.55
Standard-equivalent estimate	\$98.66	\$115.78

The important point is not merely that the recovery run cost more. The accuracy declined while cost and noise increased. This pattern suggests that the recovery changes added execution entropy without adding enough reasoning or verification quality to compensate.

6.3 Task-level movement

Task-level comparison covered 53 common tasks. To preserve benchmark confidentiality, only aggregate transition counts are reported. The recovery configuration produced six correct-to-wrong transitions and four wrong-to-correct transitions. Twenty-three tasks remained correct and twenty tasks remained wrong.

Table 3: Aggregate task-level movement from frozen baseline to recovery run.

Transition type	Count
Correct to wrong	6
Wrong to correct	4
Same correct	23
Same wrong	20

The net movement was negative. Error analysis showed stronger regression signals around code-and-document tasks, document-and-image tasks, numeric/date answers, and no-attachment semantic research tasks. Those categories point toward extraction, normalization, and evidence reconciliation issues rather than simply insufficient retries.

6.4 Smoke-run replication checks

The smoke runs were used as randomized gates. They were not treated as proof runs. The first post-patch randomized smoke achieved 12/20, or 60.00%, with zero missing finals and zero direct-solver hits. It was frozen as a positive smoke but not as evidence of full-set improvement. The next randomized smoke achieved 11/20, or 55.00%, and had higher traceback and timeout noise.

Table 4: Randomized 20-task Level-1 smoke checks.

Metric	Positive smoke	Subsequent smoke
Correct / total	12 / 20	11 / 20
Accuracy	60.00%	55.00%
Missing finals	0	0
Total attempts	24	23
Elapsed time	2819.61 s	2761.30 s
Seed	1778077117	1778082895

This sequence supports the full-run gate. The positive smoke was encouraging, but the subsequent smoke did not clear the threshold. Therefore, the appropriate decision was to avoid launching another full Level-1 run, and to avoid moving to Level-2 or Level-3 until Level-1 improves cleanly.

7 Failure Analysis

The negative ablation exposed several generic failure modes.

7.1 Retry amplification

Retries are useful when failures are transient. They are harmful when the underlying error is deterministic, such as an unavailable provider, a missing file path, an execution timeout, or a

command that repeatedly exits with a nonzero status. In those cases, retrying consumes budget and increases latency without changing the evidence available to the model.

7.2 Timeout accumulation

Timeout mentions were high in both full runs and increased in the recovery configuration. Timeout accumulation is especially damaging in benchmark settings because it reduces the number of useful tool observations per unit cost. It can also push the agent toward final-answer synthesis under time pressure, which increases the risk of poorly verified answers.

7.3 Attachment and evidence routing

Attachment-heavy tasks require local extraction before broad search escalation. The error clusters suggest that routing should distinguish between incomplete local evidence and a genuinely open-ended research requirement. An attachment should not automatically trigger expensive planner escalation if deterministic parsing can answer the question.

7.4 Final answer normalization

Several regressions skewed toward numeric/date answer shapes. This suggests that answer canonicalization should be treated as a separate reliability component. Numeric, date, unit, and list answers should be normalized and checked against the extracted evidence before final submission.

8 Recovery v2 Design Implications

The negative ablation motivates a gated orchestration policy in which planner escalation is treated as a cost-bearing intervention rather than a default behavior. Recovery v2 should therefore reduce orchestration entropy rather than add more reasoning depth by default. The design should cap retries per task and tool, stop recursive planner expansion, route simple tasks through cheaper clean-evaluation paths, require deterministic extraction before final answers, reconcile evidence before answer synthesis, fail closed on noisy tools, and log orchestration depth and entropy as telemetry rather than as optimization targets.

The next comparison should be preregistered as Baseline versus Recovery v1 versus Recovery v2 under identical seeds, prompts, routing policies, tools, and scoring code. Recovery v2 should count as successful only if accuracy is higher than Baseline, accuracy is preserved or improved versus Recovery v1, and tracebacks, timeouts, tool failures, attempts, token volume, and estimated cost are lower than Recovery v1, without increasing missing finals or baseline-relative correct-to-wrong movement. A higher accuracy number with doubled cost, exploding attempts, more timeouts, or hidden failures should be treated as another unstable recovery rather than as progress.

9 Discussion

The results support three practical conclusions.

First, small smoke-run gains are not sufficient evidence for full-set improvement. A 20-task smoke can discover promising reliability changes, but it can also overrepresent an easier or more compatible slice of the benchmark. A full-set run should require both accuracy improvement and lower operational noise.

Second, autonomy is not the same as unbounded orchestration. An agent that spawns more planning, retries more tools, and searches more broadly can become less reliable. Autonomy should include the ability to stop, choose a simpler path, and declare a tool failure non-retryable when further calls are unlikely to help.

Third, negative ablations should be preserved. They prevent misleading progress claims and reveal which changes increase cost without improving final answers. For ChromaFlow, the negative ablation suggests that the next improvements should target deterministic extraction, answer normalization, timeout policy, browser provider fallback, and evidence-backed planner escalation before additional full benchmark runs.

10 Reproducibility and Artifact Policy

The evaluation artifacts are structured as run directories containing sanitized summaries, status files, aggregate metrics, and audit manifests. Public reports intentionally avoid task text, gold answers, raw predictions, raw attachments, and detailed traces. This protects benchmark integrity while still allowing aggregate claims to be audited.

The recommended artifact policy for future public release is:

- publish aggregate metrics and non-identifying transition counts;
- publish code changes that are generic and not task-specific;
- keep private any benchmark task text, gold answers, predictions, and raw logs that could reveal tasks;
- report cost estimates as operational estimates rather than billing statements;
- label smoke runs as diagnostic unless a full-set run confirms the trend.

11 Limitations and Threats to Validity

This report has several limitations. The main full-run analysis is limited to GAIA Level-1 validation tasks. It does not establish superiority across GAIA Level-2 or Level-3, and it does not claim a top leaderboard position. The ChromaFlow implementation also contains proprietary orchestration components that are described at a high level rather than released in full.

The strongest threat to validity is external generalization. The result is one framework, one benchmark level, and one 53-task validation split. It should not be read as evidence that orchestration is generally harmful, or that a different agent, model, provider, benchmark level, or tool stack would produce the same movement. The claim is narrower: in this controlled ChromaFlow comparison, expanded orchestration reduced full-set Level-1 accuracy and increased operational noise.

The cost values are derived from campaign logs and should not be interpreted as provider billing records. They exclude potential VM, search/API, and provider-side adjustments. The operational-noise counters are also log-derived and should be read as comparative telemetry rather than exact causal labels.

Another threat is measurement sensitivity. Traceback, timeout, and tool-failure counts come from logs, so they can be affected by logging verbosity and error wording. The paper therefore uses them as relative operational signals rather than as exact causal measurements. Benchmark results are also sensitive to model behavior, tool-provider availability, web conditions, and runtime policy.

This is one reason the paper emphasizes frozen protocols, audit manifests, smoke-run gates, and negative-ablation reporting.

12 Recommendations

Based on the negative ablation, ChromaFlow should not launch Level-2 or Level-3 evaluation from the recovery configuration. The next engineering work should be generic rather than task-specific:

- reduce browser-provider fallback noise;
- make deterministic tool failures non-retryable;
- enforce process cleanup after Python and shell timeouts;
- improve document, spreadsheet, image, and attachment evidence routing;
- add numeric/date/list answer canonicalization;
- require randomized smoke runs to clear both accuracy and noise gates before any full run.

Before any new full run, the benchmark protocol should be frozen in a manifest that records the git commit, dirty diff hash, policy and prompt hashes, scorer hash, task manifest hash, seed, model label, runtime environment, abort rules, and comparison metrics. Individual failed tasks should not be rerun, and any mid-run change to prompts, retries, routing, tools, extraction logic, or scoring should invalidate the comparison.

This path preserves evaluation integrity while making the agent more useful in practice. The goal is not to tune for a fixed task list. The goal is to reduce generic execution failure modes that plausibly affect many tool-augmented reasoning tasks.

13 Conclusion

This paper presented ChromaFlow as an operational case study in autonomous reasoning reliability. The frozen full Level-1 baseline achieved 54.72%, while the expanded recovery configuration achieved 50.94% and generated more operational noise and higher cost estimates. The result is a negative ablation, not an improvement claim.

The broader lesson is that agentic capability must be evaluated together with operational behavior. Planner depth, retries, tool diversity, and multi-agent coordination can help, but they can also amplify instability. Reliable agent systems need bounded escalation, deterministic extraction, explicit non-retryable failure handling, evidence reconciliation, and honest run gates. For ChromaFlow, future progress should be measured by clean full-set gains over the 54.72% Level-1 baseline with lower noise and no compromise to benchmark integrity.

References

- [1] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, “GAIA: a benchmark for General AI Assistants,” *arXiv preprint arXiv:2311.12983*, 2023.
- [2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” *International Conference on Learning Representations*, 2023.

- [3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [4] X. Liu et al., “AgentBench: Evaluating LLMs as Agents,” *arXiv preprint arXiv:2308.03688*, 2023.
- [5] S. Zhou et al., “WebArena: A Realistic Web Environment for Building Autonomous Agents,” *arXiv preprint arXiv:2307.13854*, 2023.
- [6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” *arXiv preprint arXiv:2310.06770*, 2023.
- [7] T. Xie et al., “OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments,” *arXiv preprint arXiv:2404.07972*, 2024.
- [8] A. Drouin et al., “WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks?” *arXiv preprint arXiv:2403.07718*, 2024.
- [9] S. Yao et al., “Tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains,” *arXiv preprint arXiv:2406.12045*, 2024.
- [10] Y. Jiang, D. Li, H. Deng, B. Ma, X. Wang, Q. Wang, et al., “SoK: Agentic Skills—Beyond Tool Use in LLM Agents,” *arXiv preprint arXiv:2602.20867*, 2026.
- [11] A. Ahn, S. Lee, H. Wang, C. Park, D. Kim, J. Roh, K. Yang, W. Jang, H. Woosung, and M. S. Kim, “OrchestrationBench: LLM-Driven Agentic Planning and Tool Use in Multi-Domain Scenarios,” *International Conference on Learning Representations*, 2026. Available: <https://openreview.net/forum?id=0ljnxmf4pc>.
- [12] Z. Dong, Z. Liu, Z. Wang, Y. Li, and Z. Ma, “The Evaluation Challenge of Agency: Reliability, Contamination, and Evolution in LLM Agents,” *TechRxiv preprint*, 2026.